

DOCUMENTATION TECHNIQUE

# RetroFit

Analyse détaillée du code, de l'historique et des mécanismes de sécurité

Boutique e-commerce vintage — React.js / Node.js / MongoDB  
Dossier analysé : `FITCopiel` (dépôt Git `presentationRETROFIT`)  
Rapport généré le 8 septembre 2026

# Sommaire

1. [Présentation générale du projet](#)
2. [Historique Git et état actuel du dépôt](#)
3. [Commandes disponibles pour faire fonctionner le projet](#)
4. [Architecture générale](#)
5. [Frontend en détail \(React / Vite\)](#)
6. [Backend en détail \(Node.js / Express\)](#)
7. [Mécanismes de sécurité mis en place](#)
8. [Points de vigilance et faiblesses identifiées](#)
9. [Conteneurisation Docker](#)
10. [Parcours détaillés pas à pas \(schémas de séquence\)](#)
11. [Synthèse](#)

# 1. Présentation générale du projet

RetroFit est un site e-commerce de vêtements vintage. Le projet est composé de deux applications séparées qui communiquent via une API HTTP :

| Élément             | Détail                                                                                                                                                                           |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend            | Dossier <code>retrofit/</code> — application React 19 servie par Vite 7, stylée avec Tailwind CSS 4                                                                              |
| Backend             | Dossier <code>server/</code> — API REST Node.js avec Express 5                                                                                                                   |
| Base de données     | MongoDB (hébergée sur MongoDB Atlas), via l'ORM Mongoose                                                                                                                         |
| Stockage des images | Cloudinary (hébergement externe des photos produits)                                                                                                                             |
| Païement en ligne   | Stripe (Checkout + Webhook de confirmation)                                                                                                                                      |
| Dépôt Git distant   | <code>github.com/brandon-dev237/presentationRETROFIT.git</code> , branche <code>main</code>                                                                                      |
| Emplacement local   | <code>FITCopie1/</code> avec deux sous-projets <code>retrofit/</code> et <code>server/</code> , chacun avec son propre <code>package.json</code> et son propre <code>.env</code> |

Le code est abondamment commenté en français directement dans les fichiers sources, avec des explications pédagogiques pour chaque fichier et chaque fonction — ce rapport s'appuie sur ces commentaires ainsi que sur une lecture complète du code pour en restituer une vue d'ensemble structurée.

# 2. Historique Git et état actuel du dépôt

## 2.1 Commits enregistrés

L'historique Git du dépôt ne contient que **2 commits** à ce jour :

| Hash                 | Date             | Auteur         | Message                                    |
|----------------------|------------------|----------------|--------------------------------------------|
| <code>a389c46</code> | 11/06/2026 07:44 | brandon-dev237 | Initial commit - Boutique vintage RetroFit |
| <code>cb38917</code> | 31/08/2026 11:41 | brandon-dev237 | Premier commit                             |

Le dépôt local est synchronisé avec la branche distante `origin/main` (« up to date »).

## 2.2 Travail effectué depuis le dernier commit (non encore validé)

Une quantité importante de travail a été réalisée depuis le dernier commit (31/08/2026) mais n'a pas encore été enregistrée par un `git commit` . L'état actuel du dossier de travail montre :

**25 fichiers modifiés** (947 lignes ajoutées, 810 supprimées), notamment :

- `App.jsx` — ajout des routes des pages légales et d'un loader d'authentification pour éviter les redirections prématurées
- `assets.js` — refonte quasi complète (904 lignes de différence) du catalogue d'assets/données de démonstration
- `AppContext.jsx` — ajout de la gestion des états de chargement d'authentification ( `isAuthLoading` , `isSellerLoading` )
- `server.js` , `userRoute.js` , `sellerRoute.js` , `cartRoute.js` — intégration du limiteur de débit (rate limiting) et ajustements de sécurité des cookies
- `cartController.js` , `Address.js` , `User.js` — corrections de sécurité (utilisation de `req.userId` au lieu de données du corps de requête)

**9 nouveaux fichiers non suivis** (jamais ajoutés à Git) :

| Fichier                                           | Rôle                                                                        |
|---------------------------------------------------|-----------------------------------------------------------------------------|
| <code>server/middlewares/rateLimiter.js</code>    | Nouveau middleware anti brute-force (voir section sécurité)                 |
| <code>retrofit/src/pages/LegalNotice.jsx</code>   | Nouvelle page « Mentions légales »                                          |
| <code>retrofit/src/pages/PrivacyPolicy.jsx</code> | Nouvelle page « Politique de confidentialité » (RGPD)                       |
| <code>retrofit/Dockerfile</code>                  | Image Docker de développement pour le frontend                              |
| <code>server/Dockerfile</code>                    | Image Docker de développement pour le backend                               |
| <code>docker-compose.yml</code>                   | Orchestration des deux conteneurs (frontend + backend)                      |
| <code>retrofit/.dockerignore</code>               | Exclusions pour la construction de l'image frontend                         |
| <code>server/.dockerignore</code>                 | Exclusions pour la construction de l'image backend                          |
| <code>.claude/</code>                             | Configuration locale de l'assistant Claude Code (non applicable au produit) |

En résumé : le dernier commit Git date du 31 août 2026, mais le projet a continué d'évoluer activement après cette date (ajout du rate limiting, de la conteneurisation Docker, des pages légales RGPD et de plusieurs corrections de sécurité). Rien de tout cela n'est encore versionné — un nouveau commit serait recommandé pour ne pas perdre ce travail.

### 3. Commandes disponibles pour faire fonctionner le projet

Ces commandes correspondent aux scripts définis dans les fichiers `package.json` de chaque sous-projet et aux fichiers Docker ajoutés récemment. Elles constituent le cycle de vie standard du projet (installation, développement, production, tests).

#### 3.1 Frontend ( `retrofit/` )

| Commande                     | Rôle                                                                                                                                                    |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>npm install</code>     | Installe toutes les dépendances listées dans <code>package.json</code> (React, Vite, Tailwind, axios, react-router-dom, react-hot-toast, Playwright...) |
| <code>npm run dev</code>     | Démarre le serveur de développement Vite avec rechargement à chaud (HMR) sur <code>http://localhost:5173</code>                                         |
| <code>npm run build</code>   | Compile et optimise l'application pour la production (fichiers statiques minifiés dans <code>dist/</code> )                                             |
| <code>npm run preview</code> | Sert localement le build de production pour le tester avant déploiement                                                                                 |
| <code>npm run lint</code>    | Analyse le code avec ESLint pour détecter les erreurs de style et les mauvaises pratiques React                                                         |

#### 3.2 Backend ( `server/` )

| Commande                    | Rôle                                                                                                                                        |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| <code>npm install</code>    | Installe les dépendances serveur (Express, Mongoose, bcryptjs, jsonwebtoken, cors, cookie-parser, stripe, cloudinary, multer, dotenv)       |
| <code>npm start</code>      | Démarre le serveur avec <code>node server.js</code> (mode production, sans rechargement automatique)                                        |
| <code>npm run server</code> | Démarre le serveur avec <code>nodemon server.js</code> (redémarrage automatique à chaque modification de fichier, utilisé en développement) |

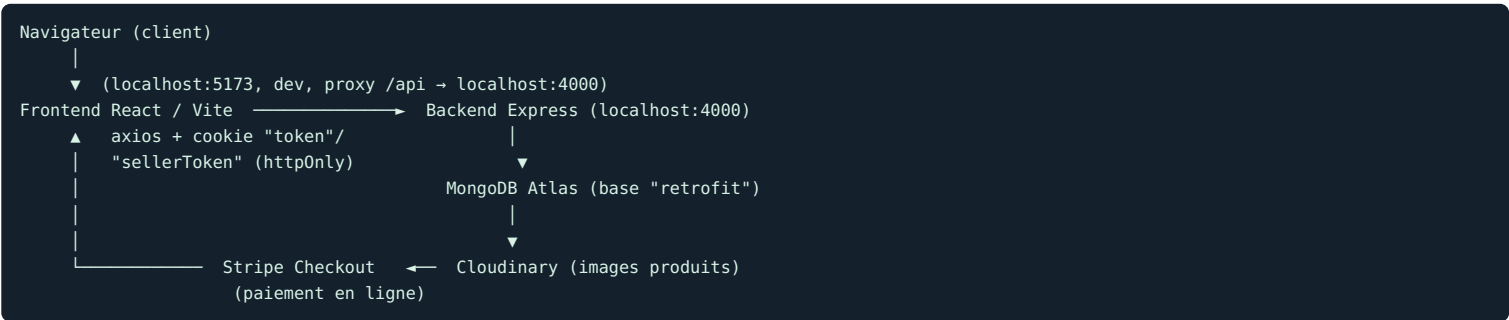
#### 3.3 Conteneurs Docker (nouvellement ajoutés, non commités)

| Commande                       | Rôle                                                                                                                                                         |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>docker compose up</code> | Construit et démarre les deux conteneurs (frontend sur le port 5173, backend sur le port 4000) avec code source monté en volume pour le rechargement à chaud |

#### 3.4 Tests (Playwright)

Le fichier `playwright.config.js` configure des tests de bout en bout (end-to-end) qui démarrent automatiquement le serveur de développement ( `npm run dev` ) et exécutent les scénarios de test présents dans un dossier `tests/` sur navigateur Chromium.

## 4. Architecture générale



En développement, Vite fait office de *proxy* : toute requête vers `/api/*` émise par le frontend est automatiquement redirigée vers le backend ( `localhost:4000` ou le conteneur `backend` sous Docker). Cela permet aux requêtes de rester « same-origin » du point de vue du navigateur et d'éviter des complications liées au CORS pendant le développement. En production, le frontend cible directement l'URL du backend déployé via la variable `VITE_BACKEND_URL` .

## 5. Frontend en détail (React / Vite)

#### 5.1 Point d'entrée et routage

| Fichier                   | Rôle                                                                                                                                                                              |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/main.jsx</code> | Point d'entrée de l'application : initialise React, englobe l'app dans <code>BrowserRouter</code> (navigation sans rechargement) et <code>AppContextProvider</code> (état global) |
| <code>src/App.jsx</code>  | Déclare toutes les routes de l'application via <code>react-router-dom</code> et affiche/masque la Navbar et le Footer selon qu'on est sur l'espace public ou l'espace vendeur     |

Table de routage définie dans `App.jsx` :

| Route                   | Composant affiché                              | Accès                                                    |
|-------------------------|------------------------------------------------|----------------------------------------------------------|
| /                       | Home                                           | Public                                                   |
| /products               | AllProducts                                    | Public                                                   |
| /products/:category/:id | ProductDetails                                 | Public                                                   |
| /products/:category     | ProductCategory                                | Public                                                   |
| /cart                   | Cart                                           | Public (mais commander requiert une connexion)           |
| /add-address            | AddAddress                                     | Connexion utilisateur requise, redirection sinon         |
| /my-orders              | MyOrders                                       | Connexion utilisateur requise, redirection sinon         |
| /loader                 | Loader                                         | Page de confirmation après paiement Stripe               |
| /mentions-legales       | LegalNotice                                    | Public                                                   |
| /confidentialite        | PrivacyPolicy                                  | Public                                                   |
| /seller/*               | SellerLayout (AddProduct, ProductList, Orders) | Connexion vendeur requise, sinon formulaire de connexion |
| *                       | Redirection vers /                             | Toute URL inconnue                                       |

5.2 Contexte global — `src/context/AppContext.jsx`

Ce fichier centralise l'état partagé par toute l'application (utilisateur connecté, panier, produits, recherche) via l'API React Context, évitant de faire transiter ces données manuellement de composant en composant.

| Fonction / donnée                                                                                                | Rôle                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>fetchUser()</code>                                                                                         | Interroge <code>GET /api/user/is-auth</code> au démarrage pour restaurer la session utilisateur à partir du cookie                            |
| <code>fetchSeller()</code>                                                                                       | Interroge <code>GET /api/seller/is-auth</code> pour restaurer la session vendeur                                                              |
| <code>fetchProducts()</code>                                                                                     | Charge le catalogue réel depuis <code>GET /api/product/list</code> (sinon conserve des produits de démonstration)                             |
| <code>addToCart</code> / <code>removeFromCart</code> / <code>deleteFromCart</code> / <code>updateCartItem</code> | Gèrent les quantités d'articles dans le panier (état local + persistance dans <code>localStorage</code> )                                     |
| <code>getCartCount()</code> / <code>getCartAmount()</code>                                                       | Calculent respectivement le nombre total d'articles et le montant total du panier                                                             |
| <code>clearCart()</code>                                                                                         | Vide le panier en mémoire et dans <code>localStorage</code> (utilisé après une commande ou un paiement réussi)                                |
| <code>isAuthLoading</code> / <code>isSellerLoading</code>                                                        | Indicateurs empêchant les routes protégées de rediriger à tort vers l'accueil avant que la vérification de session (via cookie) soit terminée |

Une configuration importante est faite ici : `axios.defaults.withCredentials = true`, qui force l'envoi automatique des cookies d'authentification avec chaque requête vers l'API — sans cela, le serveur ne pourrait jamais identifier l'utilisateur connecté.

5.3 Composants réutilisables ( `src/components/` )

| Composant                           | Rôle                                                                                                                                                         |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Navbar.jsx</code>             | Barre de navigation supérieure : logo, liens, recherche, icône panier avec badge de quantité, menu de connexion/déconnexion, menu mobile                     |
| <code>Login.jsx</code>              | Fenêtre modale de connexion/inscription utilisateur, bascule entre les deux modes et appelle <code>/api/user/login</code> ou <code>/api/user/register</code> |
| <code>Footer.jsx</code>             | Pied de page avec logo, liens de navigation par colonnes et liens vers les pages légales                                                                     |
| <code>MainBanner.jsx</code>         | Bannière d'accueil (hero) avec slogan et boutons d'appel à l'action                                                                                          |
| <code>Cartegories.jsx</code>        | Grille cliquable des catégories de vêtements, redirige vers <code>/products/:category</code>                                                                 |
| <code>BestSeller.jsx</code>         | Affiche les 5 premiers produits en stock sur la page d'accueil (« Tendances du moment »)                                                                     |
| <code>BottomBanner.jsx</code>       | Bandeau listant les arguments de vente (livraison, qualité...)                                                                                               |
| <code>Newsletter.jsx</code>         | Formulaire d'inscription à la newsletter (non connecté à un service d'emailing pour l'instant)                                                               |
| <code>ProductCard.jsx</code>        | Carte produit réutilisée partout (accueil, listes, catégories) : image, prix, notation, bouton ajouter/quantité au panier                                    |
| <code>seller/SellerLogin.jsx</code> | Formulaire de connexion réservé au vendeur, appelle <code>/api/seller/login</code>                                                                           |

5.4 Pages ( `src/pages/` )

| Page                  | Rôle                                                                  |
|-----------------------|-----------------------------------------------------------------------|
| <code>Home.jsx</code> | Assemble les composants de la page d'accueil dans l'ordre d'affichage |

|                         |                                                                                                                                                       |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| AllProducts.jsx         | Affiche tout le catalogue, filtré selon le texte tapé dans la recherche                                                                               |
| ProductCategory.jsx     | Filtre et affiche les produits d'une seule catégorie                                                                                                  |
| ProductDetails.jsx      | Fiche produit complète : galerie d'images, prix, description, produits similaires                                                                     |
| Cart.jsx                | Panier : liste des articles, calcul sous-total/taxe (2%)/total, sélection d'adresse, choix du mode de paiement (COD ou Stripe) et passage de commande |
| AddAddress.jsx          | Formulaire de saisie d'une nouvelle adresse de livraison, envoyé à <code>/api/address/add</code>                                                      |
| MyOrders.jsx            | Historique des commandes de l'utilisateur connecté ( <code>/api/order/user</code> )                                                                   |
| Loader.jsx              | Page de transition affichée après un paiement Stripe réussi : vide le panier et redirige vers les commandes après 3 secondes                          |
| LegalNotice.jsx         | Mentions légales (éditeur, hébergeur, propriété intellectuelle, médiation) — conformité LCEN                                                          |
| PrivacyPolicy.jsx       | Politique de confidentialité RGPD : données collectées, finalités, durée de conservation, droits des utilisateurs                                     |
| seller/AddProduct.jsx   | Formulaire vendeur pour ajouter un produit (jusqu'à 4 images, nom, description, catégorie, prix), envoi en <code>multipart/form-data</code>           |
| seller/ProductList.jsx  | Tableau de tous les produits côté vendeur avec interrupteur « en stock / épuisé »                                                                     |
| seller/Orders.jsx       | Liste de toutes les commandes clients (vue vendeur)                                                                                                   |
| seller/SellerLayout.jsx | Gabarit commun de l'espace vendeur (barre du haut + menu latéral + zone de contenu)                                                                   |

## 5.5 Fichiers de configuration

| Fichier              | Rôle                                                                                                                |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| vite.config.js       | Configure le plugin React, Tailwind CSS et un proxy de développement qui redirige <code>/api</code> vers le backend |
| eslint.config.js     | Règles de qualité de code (React Hooks, React Refresh, variables inutilisées)                                       |
| playwright.config.js | Configuration des tests de bout en bout sur navigateur Chromium                                                     |
| index.html           | Page HTML unique de l'application (SPA), point de montage <code>#root</code>                                        |

# 6. Backend en détail (Node.js / Express)

## 6.1 server.js — point d'entrée

Ce fichier assemble toute l'application dans l'ordre suivant :

- Connexion à MongoDB ( `connectDB()` ) et à Cloudinary ( `connectCloudinary()` )
- Middleware `cookie-parser` — lit les cookies entrants
- Middleware `cors` — restreint les origines autorisées et active l'envoi des cookies cross-origin
- Middleware conditionnel de parsing JSON — exclut spécifiquement la route du webhook Stripe qui a besoin du corps brut non transformé pour vérifier la signature
- Montage des 6 routeurs, chacun préfixé par son propre chemin ( `/api/user` , `/api/seller` , `/api/product` , `/api/cart` , `/api/address` , `/api/order` )
- Démarrage du serveur sur le port 4000 (ou celui défini dans `.env` )

## 6.2 Fichiers de configuration ( server/configs/ )

| Fichier       | Rôle                                                                                                                                                                                                         |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| db.js         | Connexion Mongoose à MongoDB Atlas. Force l'usage de résolveurs DNS publics (8.8.8.8, 1.1.1.1) pour contourner les réseaux qui bloquent les requêtes DNS SRV nécessaires aux URI <code>mongodb+srv://</code> |
| cloudinary.js | Configure le SDK Cloudinary avec les identifiants secrets ( <code>cloud_name</code> , <code>api_key</code> , <code>api_secret</code> ) issus du <code>.env</code>                                            |
| multer.js     | Configure Multer en <code>memoryStorage</code> : les fichiers uploadés (images produits) sont gardés en mémoire vive puis transmis directement à Cloudinary, sans jamais toucher le disque du serveur        |

## 6.3 Modèles de données ( server/models/ )

| Modèle     | Champs principaux                                                                | Rôle                                                                                                      |
|------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| User.js    | name, email (unique), password (haché), cartItems                                | Compte client. <code>minimize:false</code> garantit qu'un panier vide <code>{}</code> est bien enregistré |
| Product.js | name, description, price, offerPrice, image[], category, inStock                 | Fiche produit du catalogue                                                                                |
| Order.js   | userId, items[{product, quantity}], amount, address, status, paymentType, isPaid | Commande passée par un client, avec horodatage automatique ( <code>timestamps</code> )                    |
| Address.js |                                                                                  |                                                                                                           |

|  |                                                                                  |                                                 |
|--|----------------------------------------------------------------------------------|-------------------------------------------------|
|  | userId, firstName, lastName, email, street, city, state, zipcode, country, phone | Adresse de livraison rattachée à un utilisateur |
|--|----------------------------------------------------------------------------------|-------------------------------------------------|

6.4 Middlewares ( `server/middlewares/` )

| Middleware                  | Rôle                                                                                                                                                                |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>authUser.js</code>    | Vérifie la présence et la validité du cookie <code>token</code> (JWT). Si valide, injecte <code>req.userId</code> pour les contrôleurs suivants ; sinon renvoie 401 |
| <code>authSeller.js</code>  | Vérifie le cookie <code>sellerToken</code> et que l'email encodé dans le jeton correspond à <code>SELLER_EMAIL</code>                                               |
| <code>rateLimiter.js</code> | Fabrique de middleware limitant le nombre de requêtes par adresse IP sur une fenêtre de temps glissante (protection anti brute-force), détaillé en section 7        |

6.5 Contrôleurs et routes — détail méthode par méthode

userController.js / userRoute.js — comptes clients

| Route                                | Middleware(s)            | Fonction                | Rôle                                                                                                                                                    |
|--------------------------------------|--------------------------|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST <code>/api/user/register</code> | <code>authLimiter</code> | <code>register()</code> | Crée un compte : vérifie les champs, vérifie l'unicité de l'email, hache le mot de passe (bcrypt), crée l'utilisateur, émet un JWT et le pose en cookie |
| POST <code>/api/user/login</code>    | <code>authLimiter</code> | <code>login()</code>    | Vérifie l'email puis compare le mot de passe fourni au hash stocké ( <code>bcrypt.compare</code> ), émet un JWT en cas de succès                        |
| GET <code>/api/user/is-auth</code>   | <code>authUser</code>    | <code>isAuth()</code>   | Renvoie les informations du compte (sans le mot de passe, via <code>.select("-password")</code> ) si le jeton est valide                                |
| POST <code>/api/user/logout</code>   | —                        | <code>logout()</code>   | Supprime le cookie <code>token</code> du navigateur                                                                                                     |

sellerControler.js / sellerRoute.js — compte vendeur (administrateur unique)

| Route                                | Middleware(s)                   | Fonction                    | Rôle                                                                                                                                                                        |
|--------------------------------------|---------------------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST <code>/api/seller/login</code>  | <code>sellerLoginLimiter</code> | <code>sellerLogin()</code>  | Compare l'email et le mot de passe envoyés à <code>SELLER_EMAIL</code> / <code>SELLER_PASSWORD</code> définis dans <code>.env</code> , émet un JWT <code>sellerToken</code> |
| GET <code>/api/seller/is-auth</code> | <code>authSeller</code>         | <code>isSellerAuth()</code> | Confirme que le jeton vendeur est valide                                                                                                                                    |
| GET <code>/api/seller/logout</code>  | —                               | <code>sellerLogout()</code> | Supprime le cookie <code>sellerToken</code>                                                                                                                                 |

productControler.js / productRoute.js — catalogue

| Route                                | Middleware(s)                                                 | Fonction                   | Rôle                                                                                                                             |
|--------------------------------------|---------------------------------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| POST <code>/api/product/add</code>   | <code>upload.array('images')</code> , <code>authSeller</code> | <code>addProduct()</code>  | Réceptionne jusqu'à plusieurs images en mémoire, les envoie sur Cloudinary, puis crée le produit en base avec les URLs renvoyées |
| GET <code>/api/product/list</code>   | —                                                             | <code>productList()</code> | Renvoie tout le catalogue (route publique)                                                                                       |
| POST <code>/api/product/id</code>    | —                                                             | <code>productById()</code> | Renvoie un produit précis après validation du format de son identifiant Mongo                                                    |
| POST <code>/api/product/stock</code> | <code>authSeller</code>                                       | <code>changeStock()</code> | Bascule le statut « en stock / épuisé » d'un produit                                                                             |

cartController.js / cartRoute.js — panier serveur

| Route                              | Middleware(s)         | Fonction                  | Rôle                                                                                                                                                       |
|------------------------------------|-----------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST <code>/api/cart/update</code> | <code>authUser</code> | <code>updateCart()</code> | Sauvegarde le panier de l'utilisateur <b>identifié par le jeton</b> ( <code>req.userId</code> ), jamais par une valeur envoyée dans le corps de la requête |

addressController.js / addressRoute.js — adresses de livraison

| Route                              | Middleware(s)         | Fonction                  | Rôle                                                                 |
|------------------------------------|-----------------------|---------------------------|----------------------------------------------------------------------|
| POST <code>/api/address/add</code> | <code>authUser</code> | <code>addAddress()</code> | Enregistre une nouvelle adresse rattachée à l'utilisateur connecté   |
| GET <code>/api/address/get</code>  | <code>authUser</code> | <code>getAddress()</code> | Renvoie uniquement les adresses appartenant à l'utilisateur connecté |

orderController.js / orderRoute.js — commandes et paiement

| Route | Middleware(s) | Fonction | Rôle |
|-------|---------------|----------|------|
|-------|---------------|----------|------|

|                                |               |                    |                                                                                                                                         |
|--------------------------------|---------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| POST /api/order/cod            | authUser      | placeOrderCOD()    | Calcule le montant réel côté serveur (jamais celui envoyé par le client), crée une commande « paiement à la livraison »                 |
| POST /api/order/stripe         | authUser      | placeOrderStripe() | Crée la commande (non payée), génère une session Stripe Checkout et renvoie l'URL de paiement sécurisée                                 |
| POST /api/order/stripe/webhook | express.raw() | stripeWebhook()    | Reçoit la notification serveur-à-serveur de Stripe après paiement, vérifie la signature cryptographique, marque la commande comme payée |
| GET /api/order/user            | authUser      | getUserOrders()    | Renvoie uniquement les commandes de l'utilisateur connecté                                                                              |
| GET /api/order/seller          | authSeller    | getAllOrders()     | Renvoie toutes les commandes de la boutique (tableau de bord vendeur)                                                                   |

## 7. Mécanismes de sécurité mis en place

Cette section détaille, un par un, tous les dispositifs de sécurité identifiés dans le code.

### 7.1 Hachage des mots de passe (bcrypt) Authentification

Les mots de passe des comptes clients ne sont jamais stockés en clair. La fonction `register()` utilise `bcrypt.hash(password, 10)` (bibliothèque `bcryptjs`) pour transformer le mot de passe en une empreinte irréversible avant de l'enregistrer en base. À la connexion, `bcrypt.compare()` vérifie le mot de passe saisi contre ce hash sans jamais le déchiffrer. Ainsi, même en cas de fuite de la base de données, les mots de passe réels des clients restent protégés.

### 7.2 Authentification par jetons JWT (JSON Web Token) Authentification

Après une connexion réussie, le serveur génère un jeton signé cryptographiquement avec `jsonwebtoken` (`jwt.sign(payload, JWT_SECRET, {expiresIn: '7d'})`). Deux jetons distincts existent : `token` pour les clients (contient l'ID utilisateur) et `sellerToken` pour le vendeur (contient son email), ce qui empêche toute confusion entre les deux rôles. Chaque jeton expire automatiquement après 7 jours.

### 7.3 Cookies `httpOnly`, `secure` et `sameSite` Transport

Les jetons ne sont jamais exposés au JavaScript du navigateur ni stockés dans `localStorage` : ils sont transmis dans des cookies avec les attributs suivants :

- `httpOnly: true` — rend le cookie inaccessible à un script JavaScript malveillant (protection contre le vol de session par une faille XSS)
- `secure: true` en production — le cookie n'est envoyé que sur une connexion HTTPS chiffrée
- `sameSite: 'strict'` en développement et `'none'` en production — limite l'envoi du cookie aux requêtes provenant du même site, une protection contre les attaques CSRF (falsification de requête intersite)
- `maxAge: 7 jours` — durée de vie du cookie alignée sur celle du jeton

### 7.4 Middlewares de contrôle d'accès — `authUser` / `authSeller` Autorisation

Toute route sensible (panier, adresses, commandes, gestion du catalogue) est protégée par l'un de ces deux middlewares, placés avant le contrôleur dans la chaîne de traitement Express. Ils lisent le cookie, vérifient la signature du jeton avec `jwt.verify()`, et coupent immédiatement la requête (401 Non autorisé) si le jeton est absent, expiré ou falsifié. Le vendeur bénéficie d'une vérification supplémentaire : l'email contenu dans le jeton doit correspondre exactement à `SELLER_EMAIL`.

### 7.5 Identification par jeton plutôt que par les données du client Anti-IDOR

Point de sécurité important repéré dans le code (avec un commentaire explicite de l'auteur dans `cartController.js`) : l'identifiant de l'utilisateur utilisé pour modifier le panier, ajouter une adresse ou consulter ses commandes provient **toujours** de `req.userId` (injecté par le middleware après vérification du jeton), et jamais d'un champ envoyé dans le corps de la requête. Cela empêche un utilisateur malveillant de manipuler les données d'un autre compte en falsifiant une requête (attaque de type IDOR — Insecure Direct Object Reference).

### 7.6 CORS restreint Réseau

Le serveur n'autorise que les origines explicitement listées dans `allowedOrigins` (`http://localhost:5173` en développement) à communiquer avec lui, avec `credentials: true` pour permettre l'envoi des cookies. Toute autre origine se voit refuser l'accès par le navigateur.

### 7.7 Limitation de débit / anti brute-force (`rateLimiter.js`) Nouveau

Middleware ajouté récemment (non encore commité) qui limite à **10 tentatives par adresse IP sur une fenêtre de 15 minutes** les appels aux routes de connexion (`/api/user/login`, `/api/user/register`, `/api/seller/login`). Au-delà de la limite, le serveur répond `429 Too Many Requests` avec un en-tête `Retry-After`. Cela freine les attaques par force brute visant à deviner un mot de passe.

### 7.8 Vérification de signature du webhook Stripe Paiement



La route `/api/order/stripe/webhook` ne fait confiance à une notification de paiement que si sa signature cryptographique est valide : `stripe.webhooks.constructEvent(req.body, signature, STRIPE_WEBHOOK_SECRET)`. Cela empêche quiconque de falsifier une confirmation de paiement en appelant directement cette route. Le corps de la requête est volontairement reçu à l'état brut (`express.raw()`), condition nécessaire à cette vérification.

### 7.9 Validation des identifiants MongoDB Robustesse

Avant toute recherche par identifiant (`productById`, `changeStock`), le code vérifie que la chaîne fournie est un `ObjectId` MongoDB valide (`mongoose.Types.ObjectId.isValid(id)`), évitant une erreur serveur ou un comportement indéfini si un identifiant malformé ou malveillant est soumis.

### 7.10 Non-exposition du mot de passe dans les réponses API Confidentialité

La fonction `isAuth()` utilise `User.findById(userId).select("-password")` : le hash du mot de passe n'est jamais renvoyé au frontend, même sous forme chiffrée, réduisant la surface d'exposition en cas d'interception réseau.

### 7.11 Secrets isolés dans des variables d'environnement Configuration

Toutes les informations sensibles (clé JWT, identifiants MongoDB, clés Stripe, clés Cloudinary, identifiants du compte vendeur) sont stockées dans des fichiers `.env` distincts pour le frontend et le backend, jamais codées en dur dans le code source. Ces fichiers sont explicitement exclus du contrôle de version par `.gitignore` et de la construction des images par `.dockerignore`, ce qui évite qu'ils ne finissent sur GitHub ou dans une image Docker publiée.

| Fichier       | Variables présentes (valeurs masquées)                                                                                                                                                     |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| retrofit/.env | VITE_CURRENCY, VITE_BACKEND_URL                                                                                                                                                            |
| server/.env   | JWT_SECRET, FRONTEND_URL, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, NODE_ENV, SELLER_EMAIL, SELLER_PASSWORD, MONGODB_URI, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET |

### 7.12 Isolation par conteneurs Docker Nouveau

Les `Dockerfile` et `docker-compose.yml` récemment ajoutés isolent chaque service (frontend/backend) dans son propre conteneur avec ses propres dépendances (`node_modules` n'est jamais partagé entre l'hôte et le conteneur, évitant les conflits de versions natives), tout en gardant les secrets dans des fichiers `.env` chargés au démarrage plutôt qu'intégrés à l'image.

### 7.13 Stockage des fichiers uploadés en mémoire, jamais sur disque Upload

Multer est configuré en `memoryStorage()` : les images envoyées par le vendeur transitent directement de la mémoire vive vers Cloudinary sans être écrites sur le disque du serveur, réduisant les risques associés au stockage de fichiers arbitraires sur le système de fichiers du serveur.

### 7.14 Conformité RGPD et information légale Nouveau

Deux nouvelles pages, non encore commitées, formalisent les obligations légales du site : `LegalNotice.jsx` (mentions légales, conformément à l'article 6-III de la LCEN) et `PrivacyPolicy.jsx` (politique de confidentialité RGPD détaillant les données collectées, leur finalité, leur durée de conservation et les droits des utilisateurs).

### 7.15 Protection XSS de base via React Frontend

React échappe par défaut tout contenu inséré dynamiquement dans le JSX (aucun usage de `dangerouslySetInnerHTML` n'a été repéré dans le code), ce qui protège nativement contre l'injection de scripts via les données affichées (noms de produits, descriptions, etc.).

## 8. Points de vigilance et faiblesses identifiées

Cette section liste des points qui ne remettent pas en cause l'architecture générale mais qui mériteraient d'être renforcés avant une mise en production réelle.

| Sujet                   | Constat                                                                                                                                                                                         | Risque                                                                                                                                                                |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mot de passe vendeur    | Dans <code>sellerController.js</code> , le mot de passe est comparé en clair ( <code>password === process.env.SELLER_PASSWORD</code> ) et stocké en clair dans <code>.env</code> , sans hachage | Si le fichier <code>.env</code> fuit, le mot de passe administrateur est immédiatement exploitable                                                                    |
| Upload de fichiers      | La configuration Multer ( <code>configs/multer.js</code> ) ne définit ni limite de taille ( <code>limits</code> ) ni filtre de type MIME ( <code>fileFilter</code> )                            | Un vendeur (ou un jeton compromis) pourrait envoyer des fichiers très volumineux ou d'un type inattendu                                                               |
| Rate limiter en mémoire | <code>rateLimiter.js</code> stocke les compteurs de tentatives dans une <code>Map</code> en mémoire du processus                                                                                | Ne fonctionne que sur une seule instance ; inefficace si le backend est répliqué sur plusieurs conteneurs/serveurs, et les compteurs sont perdus à chaque redémarrage |

|                           |                                                                                                                                                                                                                      |                                                                                                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| En-têtes de sécurité HTTP | Aucun middleware de type <code>helmet</code> n'est utilisé                                                                                                                                                           | Absence d'en-têtes comme <code>Content-Security-Policy</code> , <code>X-Frame-Options</code> , <code>X-Content-Type-Options</code> , qui renforcent la défense en profondeur |
| Validation des entrées    | Aucune bibliothèque de validation de schéma (ex. Zod, Joi, express-validator) n'encadre les corps de requête ; les contrôles sont manuels et partiels                                                                | Risque de données incohérentes ou d'erreurs non maîtrisées en base selon les champs envoyés                                                                                  |
| Journalisation            | De nombreux <code>console.log</code> affichent des données utilisateur (identifiants, adresses complètes) dans les contrôleurs (ex. <code>addressController.js</code> )                                              | Fuite d'informations personnelles dans les journaux serveur si ceux-ci sont conservés ou exposés (non conforme RGPD en l'état pour un usage en production)                   |
| CORS                      | <code>allowedOrigins</code> est codé en dur avec uniquement <code>http://localhost:5173</code> dans <code>server.js</code> , alors que <code>FRONTEND_URL</code> existe déjà en variable d'environnement pour Stripe | À adapter avant déploiement en production, sinon le frontend déployé ne pourra pas communiquer avec l'API                                                                    |
| Historique Git            | Seuls 2 commits existent, et un volume important de changements (dont plusieurs correctifs de sécurité) n'est pas encore versionné                                                                                   | Risque de perte de travail, absence de traçabilité fine des évolutions                                                                                                       |

## 9. Conteneurisation Docker

Trois fichiers, ajoutés récemment et non encore commités, mettent en place un environnement de développement conteneurisé :

```
docker-compose.yml
```

Définit deux services : `backend` (construit depuis `./server`, port 4000, variables issues de `server/.env`) et `frontend` (construit depuis `./retrofit`, port 5173, variables issues de `retrofit/.env`, avec une variable additionnelle `API_PROXY_TARGET` permettant à Vite de joindre le service backend par son nom Docker plutôt que par `localhost`). Le code source de chaque service est monté en volume, ce qui préserve le rechargement à chaud (nodemon côté backend, HMR Vite côté frontend) exactement comme en local.

```
retrofit/Dockerfile · server/Dockerfile
```

Deux images quasi identiques basées sur `node:24-alpine` (image Node.js légère) : elles copient les fichiers de dépendances, exécutent `npm install`, copient le reste du code, exposent leur port respectif (5173 / 4000) et démarrent le serveur de développement correspondant.

```
.dockerignore (x2)
```

Exclut `node_modules`, `.git`, et surtout `.env` de ce qui est envoyé au moteur Docker lors de la construction de l'image, évitant que des secrets ne soient embarqués dans l'image elle-même.

## 10. Parcours détaillés pas à pas (schémas de séquence)

Cette section reprend les cinq parcours les plus importants de l'application et détaille, étape par étape, ce qui se passe exactement dans le code — côté navigateur, côté serveur et côté services externes — depuis le clic de l'utilisateur jusqu'à la réponse finale.

### 10.1 Inscription et connexion d'un utilisateur



**Ce qu'il faut retenir** : le mot de passe en clair ne quitte jamais le formulaire sans être immédiatement haché côté serveur (jamais côté client). Le cookie `token` est posé par le navigateur automatiquement grâce à l'en-tête `Set-Cookie` de la réponse — le frontend n'a rien à faire pour le stocker, et ne peut d'ailleurs pas le lire (`httpOnly`). La connexion (`login()`) suit exactement le même schéma, à la différence que l'étape 4 devient `bcrypt.compare(password, user.password)` au lieu d'un hachage.

### 10.2 Vérification automatique de session au chargement de l'application

```

Chargement de l'app (main.jsx → App.jsx → AppContextProvider)
|
▼
useEffect(() => { fetchUser(); fetchSeller(); fetchProducts(); }, []) ← s'exécute une seule fois
|
└─ fetchUser() — GET /api/user/is-auth (cookie "token" envoyé automatiquement, withCredentials:true)
    |
    ▼ côté serveur : authUser middleware
    jwt.verify(token, JWT_SECRET) → si valide, req.userId = tokenDecode.id → next()
    si invalide/absent → 401 Not Authorized
    |
    ▼
    isAuthenticated() : User.findById(userId).select("-password") → renvoie l'utilisateur (sans mdp)
    |
    ▼
    setUser(data.user) puis setIsAuthLoading(false) [finally]
    |
    └─ pendant ce temps, App.jsx affiche sur les routes protégées
        (/add-address, /my-orders) au lieu de rediriger vers "/" à tort

```

**Pourquoi c'est important** : sans l'indicateur `isAuthLoading`, un utilisateur connecté qui recharge la page `/my-orders` serait redirigé vers l'accueil pendant la fraction de seconde où le serveur n'a pas encore répondu — c'est un piège classique du rendu côté client que le code évite explicitement (voir le commentaire dédié dans `App.jsx`).

### 10.3 Ajout au panier et calcul du total (logique `Cart.jsx` + `AppContext.jsx`)

```

Clic sur "Ajouter au panier" (ProductCard.jsx ou ProductDetails.jsx)
|
▼
addToCart(productId) [AppContext.jsx]
  setCartItems(prev => ({ ...prev, [productId]: (prev[productId] || 0) + 1 }))
  |
  ▼
useEffect déclenché par le changement de cartItems
  localStorage.setItem('cartItems', JSON.stringify(cartItems)) ← persistance navigateur

Sur la page Cart.jsx, à chaque rendu :
  cartArray = Object.keys(cartItems).map(id => products.find(p => p._id === id) + quantité)
  subtotal = Σ (offerPrice × quantité) pour chaque article
  tax = floor(subtotal × 0.02 × 100) / 100 arrondi au centime, 2%
  total = floor((subtotal + tax) × 100) / 100

```

Ce calcul est fait **uniquement pour l'affichage** côté navigateur. Le montant réellement facturé est recalculé indépendamment côté serveur au moment de la commande (section suivante) — le frontend ne fait jamais foi pour le prix final, ce qui empêche un client de manipuler le prix affiché avant paiement.

### 10.4 Passage de commande et paiement (COD vs Stripe)

```

Clic sur "Payer" (Cart.jsx → placeOrder())
|
└─ vérifications : user connecté ? adresse sélectionnée ? panier non vide ?
|
▼
Si paiement à la livraison (COD) :
  axios.post('/api/order/stripe' ... non, '/api/order/cod', {items, address})
  |
  ▼ authUser vérifie le cookie → req.userId
  placeOrderCOD() :
    1. pour chaque item : product.findById(item.product) → prix réel en base (jamais celui du client)
    2. amount = Σ(offerPrice × quantité) + taxe 2%
    3. order.create({userId, items, amount, address, paymentType:"COD"})
    4. → {success:true, message:'Order Placed Successfully'}
  frontend : clearCart() puis navigate('/my-orders')

Si paiement en ligne (Stripe) :
  axios.post('/api/order/stripe', {items, address})
  |
  ▼ authUser vérifie le cookie → req.userId
  placeOrderStripe() :
    1. recalcule le montant réel côté serveur (idem COD)
    2. order.create({... , paymentType:"Online", isPaid:false}) ← commande créée MAIS non payée
    3. construit line_items pour Stripe (nom produit + prix en centimes)
    4. stripe.checkout.sessions.create({line_items, success_url, cancel_url,
                                         metadata:{orderId, userId}})
    5. → {success:true, url: session.url}
  frontend : window.location.href = data.url ← redirection vers la page Stripe hébergée

... le client paie sa carte sur le domaine stripe.com (Retrofit ne voit jamais le numéro) ...

```

```
Stripe redirige le navigateur vers /loader?next=orders (success_url)
Loader.jsx : clearCart() puis, après 3s, navigate('/my-orders')
```

```
EN PARALLÈLE, Stripe appelle directement le serveur (pas le navigateur) :
POST /api/order/stripe/webhook (signature dans l'en-tête "stripe-signature")
stripeWebhook() :
1. stripe.webhooks.constructEvent(body_brut, signature, STRIPE_WEBHOOK_SECRET)
  → si la signature ne correspond pas, rejet immédiat (400)
2. si event.type === 'checkout.session.completed' :
  order.findByIdAndUpdate(orderId, {isPaid:true})
3. → {received:true}
```

**Point clé de sécurité** : la commande n'est marquée `isPaid:true` que par le webhook, jamais par la redirection du navigateur vers `/loader`. Un utilisateur ne peut donc pas se faire créditer une commande payée simplement en visitant l'URL de succès sans avoir réellement payé — seule une notification authentifiée par la signature secrète de Stripe peut le faire.

## 10.5 Ajout d'un produit par le vendeur (upload d'images)

```
Formulaire AddProduct.jsx (espace vendeur)
| 4 champs image (File) + nom/description/catégorie/prix
▼
FormData : productData (JSON stringifié) + images (fichiers bruts)
axios.post('/api/product/add', formData) [multipart/form-data]
|
▼ Chaîne de middlewares Express (dans l'ordre déclaré sur la route) :
1. upload.array('images') → Multer lit les fichiers en mémoire, les place dans req.files
2. authSeller → vérifie le cookie sellerToken + l'email du jeton
3. addProduct() → contrôleur exécuté seulement si 1 et 2 ont réussi
|
▼ addProduct() :
a. JSON.parse(req.body.productData) → récupère nom/description/catégorie/prix
b. si req.files vide → refus ("Veuillez ajouter au moins une image")
c. pour chaque fichier : cloudinary.uploader.upload_stream(buffer) → URL sécurisée
  (Promise.all : les envois vers Cloudinary se font en parallèle)
d. product.create({...productData, image: [urls...]})
e. → {success:true, message:"product Added"}
frontend : fetchProducts() recharge le catalogue, formulaire vidé
```

Notez l'ordre des middlewares : Multer s'exécute *avant* `authSeller`, car Multer doit d'abord parser le corps `multipart/form-data` pour que les champs texte (dont potentiellement un jeton s'il était envoyé autrement) soient disponibles ; ici l'authentification passe par le cookie donc l'ordre n'a pas d'incidence sur la sécurité, mais il est important pour que `req.files` et `req.body.productData` soient correctement peuplés avant que le contrôleur ne s'exécute.

# 11. Synthèse

RetroFit est une application e-commerce complète suivant une architecture classique frontend/backend découplée, avec une séparation nette des responsabilités : React/Vite pour l'interface, Express/MongoDB pour la logique métier et la persistance, et trois services tiers spécialisés (MongoDB Atlas, Cloudinary, Stripe) pour la base de données, le stockage d'images et le paiement.

Le socle de sécurité repose sur des mécanismes standards et reconnus du secteur : hachage bcrypt des mots de passe, authentification par JWT dans des cookies `httpOnly / secure / sameSite`, middlewares d'autorisation systématiques sur les routes sensibles, identification de l'utilisateur exclusivement via le jeton vérifié (protection anti-IDOR), CORS restreint, vérification cryptographique du webhook Stripe, et — ajout récent — une limitation de débit contre les attaques par force brute. Le projet a également commencé à intégrer les obligations légales françaises et européennes (mentions légales, RGPD) et une conteneurisation Docker pour fiabiliser l'environnement de développement.

Les points de vigilance identifiés en section 8 (mot de passe vendeur non haché, absence de limites d'upload, absence d'en-têtes de sécurité HTTP, journalisation trop verbeuse) sont des améliorations classiques à apporter avant un déploiement en production, mais ne remettent pas en cause la solidité générale de l'architecture mise en place.